ADR 0003
Camadas Service e Repository

Data	Status	Integrantes
04/03/2026	Aceito	Alexander Nogueira Caio Nunes de Abreu Cassiano Nunes de Abreu Gabriel Naoki Uto Turigoe Wyllian Mariano Nogueira

Sumário

Contexto 2
Decisão 2
Consequências 2
- Positivas 2
- Negativas / Riscos 3

Mitigações 3
Alternativas consideradas 3
Follow Up / Próximos passos 3
Referencias 3

Contexto

A aplicação utilizava a abordagem **"Controller-first"**, onde a classe ContatoController acumulava as responsabilidades de orquestração da UI, regras de negócio e persistência em arquivo texto. Embora funcional para o início do projeto, essa estrutura começou a apresentar limitações:

- O Controller tornou-se uma "Classe Deus", com muitas responsabilidades.
- Dificuldade em trocar a forma de persistência (ex: mudar de .txt para Banco de Dados) sem afetar a lógica de negócio.
- Necessidade de isolar as regras de validação complexas da manipulação direta de arquivos.

Decisão

Refatorar a aplicação para extrair as responsabilidades do Controller em duas novas camadas especializadas:

- **Camada Repository:**

```
// Interface para abstração do acesso a dados
public interface ContatoRepository {
    void save(Contato contato);
    List<Contato> findAll();
    void delete(int id);
}
```

- **Camada Service:**

```
// Camada que detém as regras de negócio e validações

public class ContatoService {

    private final ContatoRepository repository;

    // Injeção de dependência via construtor

    public ContatoService(ContatoRepository repository) {

        this.repository = repository;

    }

}
```

- **Fluxo de Dependência:** A View chama o Controller -> O Controller chama o Service -> O Service chama o Repository.

Consequências

Positivas

- **Manutenibilidade:** A persistência em arquivo texto fica isolada no FileContatoRepository, facilitando uma migração futura para SQL.
- **Testes Granulares:** Possibilidade de testar as regras de negócio (Service) usando um repositório "Mock" (em memória), sem tocar no sistema de arquivos.
- **Independência:** O Controller agora foca apenas em gerenciar o estado da tela e coordenar o SwingWorker.

Negativas / Riscos

- **Aumento da Verbosidade:** Mais classes e interfaces para gerenciar.
- **Curva de Aprendizado:** Maior complexidade arquitetural para a equipe entender o fluxo de dados entre as 4 camadas.

Mitigações

- **Injeção de Dependência:** Utilizar construtores para passar as dependências, evitando acoplamento rígido (new) dentro das classes.
- **Padronização:** Manter métodos pequenos e com nomes claros seguindo os termos do domínio (ex: registrarContato, buscarAtivos).
-

Alternativas consideradas

- Manter lógica e persistência nas Views (Rejeitado)
  - Simples inicialmente, porém mantém baixa testabilidade e alto acoplamento; não atende objetivo pedagógico.
- Extrair as camadas Service e Repository (Considerado, aceito para momento posterior)

Follow Up / Próximos passos

- Criar o pacote br.edu.fatesg.repository e br.edu.fatesg.service.
- Mover toda a lógica de java.nio.file do Controller para a implementação do Repository.
- Atualizar os diagramas C2/C3 no Documento de Arquitetura de Software (DAS).
- Implementar testes unitários focados na classe Service.

Referências

- Projeto: https://github.com/marquesclayton/ads3.contatos-camadas
- Padrão: **Data Access Object (DAO) / Repository Pattern**
- Fowler — **Strangler Fig Pattern** (para a migração das funções)